Consulta: pasos a seguir para la etapa 3 (refactoring)

Fecha: 07/11/2026
Integrante: Javier Ovelar
Herramienta: Claude Code (Anthropic), modelo Opus 5.5
Repositorio: TPI_MS2_Mazza_Morales_Ovelar_Paschetta

Pregunta

> **Javier**
> soy javier, y tengo que hacer la etapa 3, de refactoring, esta en la carpeta @TPI_MS2_Mazza_Morales_Ovelar_Paschetta\. Indicame los pasos a seguir asi lo voy haciendo

Respuesta

Claude

Estos son los pasos de la Fase 3 para tu parte (`app/routers/estados.py`: `proc_estado` y `asignar`). Salen de `PLAN_DE_TRABAJO.md`, de tu ficha de smells (`docs/diagnostico/javier_ficha_smells.md`) y de cómo lo hicieron Cristian y Gastón. No modifiqué nada.

Estado actual

- Ya tenés creada la rama `refactor/javier-estados` y estás parado en ella. Está en el mismo commit que `origin/dev` (al día) y todavía no la subiste a GitHub.
- Lo que te toca garantizar es el **Move Method**.

Paso 0: preparación

1. `git status`: tiene que decir `On branch refactor/javier-estados`.
2. `.\run.ps1 test`: la suite completa tiene que estar en verde **antes** de tocar nada.

Paso 1: aplicar los refactorings, uno por commit

Estos son los candidatos que vos mismo propusiste en la ficha, en un orden sugerido:

#	Refactoring	Técnica	Hallazgos de tu ficha
1	Pasar las transiciones permitidas (los `if` con `ok`) y la lista de estados cerrados `["RES","RECH","CANC"]` a `app/dominio.py`, como hicieron Cristian (`AREAS`) y Gastón (`REGLAS_POR_TIPO`)	**Move Method** (el obligatorio)	2, 3, 10
2	Leer las columnas por nombre (`sqlite3.Row`) en lugar de `s[5]`, `r[4]`, `est[4]`...	Igual que Cristian y Gastón	9
3	Sacar de `proc_estado` el registro en historial y la notificación a funciones propias	Extract Method	8, 11

4	Nombres expresivos (`s`, `r`, `act`, `f`)	Rename	12

Después de **cada** cambio:

1. `.\run.ps1 test` con la suite **completa**, no solo `test_estado.py` y `test_asignacion.py`.
2. Commit con prefijo, por ejemplo: `git commit -m "refactor: mover las transiciones de estado a app/dominio.py"`

> **Ojo:** un refactoring no cambia el comportamiento. Cristian dejó asentado que pasó todos los tests "sin cambiar ningún assert". Los hallazgos de prioridad alta (1 errores con 200, 4 motivo de rechazo, 5 modelo de entrada, 6 historial en reasignaciones) **cambian lo que responde el sistema** y harían fallar los tests actuales. Así que no son refactorings. Antes de tocarlos, consultalo con el grupo.

Dato de coordinación:

- `consultas.py:38` (de Bruno) también tiene escritos los estados cerrados `("RES", "RECH", "CANC")`. Si los pasás a `dominio.py`, avisale.
- La rama de Gastón (`refactor/gaston-solicitudes`, todavía sin mergear) también modifica `app/dominio.py`. Conviene que entre primero a `dev` y actualizar tu rama antes de tocar ese archivo, para no tener conflictos.

Paso 2: documentar

1. Crear `docs/refactoring/javier_refactoring.md` con el mismo formato que `cristian_refactoring.md`: una tabla resumen (refactoring, técnica, hallazgos, principio) y, por cada refactoring, el smell que resuelve, la técnica, el principio y el código **antes y después**.
2. Guardar la captura de la suite en verde en `docs/evidencias/fase3/javier/`, como hicieron Bruno y Gastón.
3. Completar el registro de IA de la fase 3 en `docs/uso-ia/javier.md` y poner las conversaciones en `anexo/ia/`.
4. Hacer el commit: `docs: refactorings de la fase 3 (antes y despues) y registro de uso de IA`.

Paso 3: subir y abrir el PR

1. `git push -u origin refactor/javier-estados`
2. Abrir en GitHub el PR **hacia** `dev`. La descripción va con la plantilla `.github/pull_request_template.md`: qué cambió y por qué.
3. El revisor es **Gastón**, según la rotación del plan.
4. Atender sus comentarios y dejar registro de lo que corregiste.

Cuando quieras arrancar con el paso 1, decime y te guío con el Move Method sobre el código concreto.